

Part 3: Build with GitHub Copilot + VS Code

Video: Build ServiceNow Apps with AI — GitHub Copilot Edition

*Same app, different AI. We build the **AI Agent Management System** using GitHub Copilot inside VS Code — autocomplete, inline suggestions, and Copilot Chat.*

Prerequisites

From Part 1, you should have:

- ■ VS Code installed
- ■ Node.js + NPM installed
- ■ ServiceNow SDK installed
- ■ GitHub Copilot + Copilot Chat extensions installed in VS Code
- ■ Signed into GitHub with an active Copilot subscription

What We're Building (Same App)

AI Agent Management System — the exact same app from Part 2:

Table	Key Fields
AI Agents	Name, Description, Status, Type, Owner, Connected CI (→ CMDB)
Agent Tools	Name, Description, Type (Tool/Sub-Agent), Parent Agent, Capabilities

Step 1: Create the Project

Open your terminal (inside VS Code: `Ctrl+``):

```
mkdir ai-agent-manager-copilot && cd ai-agent-manager-copilot
```

```
npx @servicenow/sdk init
npm install
```

Then open the project in VS Code:

```
code .
```

Step 2: Create the AI Agents Table File

Using Copilot Chat (The Fast Way)

1. Open **Copilot Chat** panel (Cmd+Shift+I on Mac, Ctrl+Shift+I on Windows)
2. Type this prompt:

```
Create a ServiceNow Fluent API table definition in TypeScript for an "AI Agent" table.
```

```
The file should import from '@servicenow/sdk/core'.
```

```
Table name: x_vendor_aiagent_ai_agent
Label: AI Agent
```

```
Fields (all prefixed with x_vendor_aiagent_):
```

```
- name: StringColumn, mandatory, maxLength 200
- description: StringColumn, maxLength 1000
- status: ChoiceColumn with choices { active: "Active", inactive: "Inactive", development: "In De
- agent_type: ChoiceColumn with choices { autonomous: "Autonomous", assisted: "Assisted", hybrid:
- owner: ReferenceColumn to sys_user
- connected_ci: ReferenceColumn to cmdb_ci
- version: StringColumn, maxLength 50
- last_active: DateTimeColumn
- documentation_url: URLColumn
```

```
Use the choices object syntax: { key: { label: "Label" } }
Export the table with the same name as the table name.
```

3. Copilot Chat will generate the code
4. Click **Insert at Cursor** or **Copy** and paste into `src/fluent/tables/ai-agent.now.ts`

Using Inline Autocomplete (The Interactive Way)

1. Create file: `src/fluent/tables/ai-agent.now.ts`
2. Start typing the import:

```
import {
  Table, StringColumn,
```

3. Copilot will suggest the remaining imports — press **Tab** to accept

4. Start the export:

```
export const x_vendor_aiagent_ai_agent = Table({
```

5. Copilot will start suggesting the full table definition

6. Accept suggestions with **Tab**, modify as needed

Tips for Better Copilot Suggestions

- **Add a comment first** — Copilot reads comments for context:

```
// ServiceNow Fluent API table for managing AI agents  
// Fields: name, description, status, type, owner, CMDB connection
```

- **Type the first field completely** — Copilot learns the pattern and generates the rest

- **Open reference files** — if you have an example **.now.ts** file open in another tab, Copilot uses it as context

Step 3: Create the Agent Tools Table

Using Copilot Chat

Open Copilot Chat and prompt:

```
Create another ServiceNow Fluent API table for "Agent Tool" in the same style.
```

```
Table name: x_vendor_aiagent_agent_tool  
Label: Agent Tool
```

```
Fields (all prefixed x_vendor_aiagent_):  
- name: StringColumn, mandatory, maxLength 200  
- description: StringColumn, maxLength 1000  
- tool_type: ChoiceColumn { tool, sub_agent, api, plugin }  
- parent_agent: ReferenceColumn to x_vendor_aiagent_ai_agent (mandatory)  
- capabilities: StringColumn, maxLength 2000  
- permissions: StringColumn, maxLength 500  
- is_active: BooleanColumn  
- execution_count: IntegerColumn
```

```
Save as src/fluent/tables/agent-tool.now.ts
```

Using Inline Autocomplete

1. Create **src/fluent/tables/agent-tool.now.ts**

2. Copy the import block from your first file

3. Start typing — since Copilot now has context from **ai-agent.now.ts**, suggestions will follow the same pattern

4. When you type `parent_agent: ReferenceColumn({`, Copilot should suggest the reference to your AI Agents table
-

Step 4: Add ACLs

Using Copilot Chat

Create ServiceNow ACLs using the Fluent API for these two tables:

- `x_vendor_aiagent_ai_agent`
- `x_vendor_aiagent_agent_tool`

Rules:

- admin role: full CRUD
- itil role: read + write
- any authenticated user: read only

Save as `src/fluent/acls/agent-acls.now.ts`

Review What Copilot Generates

Check that ACLs:

- Import from `@servicenow/sdk/core`
 - Use correct table names
 - Define proper role requirements
 - Have unique `$id` values
-

Step 5: Build and Deploy

Build

```
npm run build
# or: now-sdk build
```

If Build Fails

With Copilot Chat: Select the error output, right-click → **Copilot: Explain This**. Or paste the error into Copilot Chat:

`This build error occurred: [paste error]. How do I fix it?`

Common Copilot mistakes to watch for:

- Using `defaultValue` instead of `default` for ChoiceColumn

- Using array syntax `["Active", "Inactive"]` instead of object syntax `{ active: { label: "Active" } }`
- Missing scope prefix on column names
- Export name not matching table name

Deploy

```
now-sdk install --auth mydev
```

Step 6: Verify in ServiceNow

Same verification as Part 2:

1. Log into your instance
2. Find **AI Agent Manager** in the navigator
3. Create a test AI Agent record
4. Create a test Agent Tool linked to it
5. Link the agent to a CMDB CI

Copilot vs Claude Code: Key Differences

Aspect	Claude Code	GitHub Copilot
Interface	Terminal conversation	VS Code inline + chat panel
How you work	Describe what you want → get complete files	Start typing → accept/modify suggestions
File creation	Creates files directly on disk	Generates code you paste/insert
Error fixing	Paste error → it fixes and re-runs	Paste error in chat → get fix suggestions
Context	Reads entire project automatically	Uses open tabs + current file as context
Best for	Generating complete files from scratch	Iterating and extending existing code
Runs commands	Yes — builds, deploys, runs tests	No — you run commands yourself

Strengths	Full project awareness, agentic workflow	Tight editor integration, real-time suggestions
-----------	--	---

Copilot Power Tips for ServiceNow

- 1. Keep a reference file open** — Open an example `.now.ts` file in a tab. Copilot uses open files as context.
- 2. Use `// @copilot` comments** — Add detailed comments above your code describing what you want.
- 3. Copilot Chat workspace mode** — Use `@workspace` in Copilot Chat to ask questions about your entire project:

`@workspace` What tables are defined in this project?

- 4. Inline edits** — Select code → press `Cmd+I` → describe the change:
`Add a "priority" choice field with values high, medium, low`
- 5. Generate tests** — Right-click a file → Copilot → Generate Tests

Summary

Step	What We Did
1	Scaffolded project with SDK
2	Built AI Agents table using Copilot Chat + autocomplete
3	Built Agent Tools table with Copilot
4	Added ACLs with Copilot
5	Built and deployed
6	Verified in ServiceNow

The Copilot workflow is more hands-on — you're in the editor, guiding the AI line by line, compared to Claude Code where you describe and it builds.

Next: [Part 4 — Build with Windsurf →](#)